

KnowledgeLM (DocAI)

開發者再入門指南

Developer Re-Onboarding Guide

為一位「久未碰專案的開發者」量身設計

Project overview · Code examples · Pros & Cons · File-splitting plan

Generated 2026-05-17 · Branch: beta_skills_v01

目錄 · Table of Contents

Part 1 · 專案全景 (The Big Picture)

1.1 一句話定位

KnowledgeLM (內部代號 DocAI) 是一個「給 PDF 用的 ChatGPT」——你把一堆 PDF 餵進去，它幫你把內容拆碎、轉成向量、存起來；之後你用自然語言問問題，它會找到最相關的片段，再請 LLM 用這些片段回答你。這就是業界俗稱的 RAG (Retrieval-Augmented Generation, 檢索增強生成)。

1.2 一個比喻：圖書館員機器人

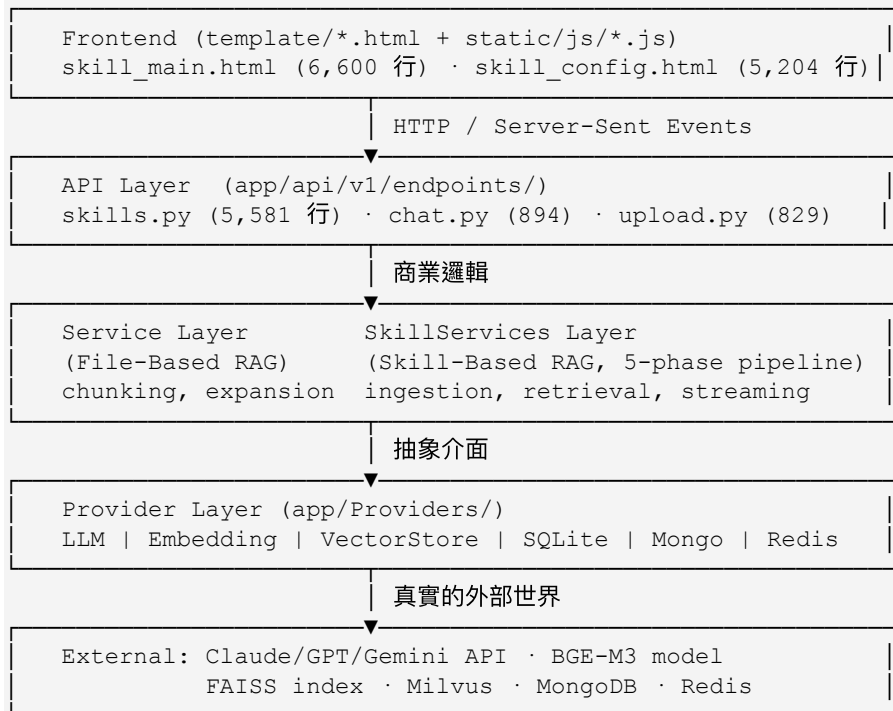
想像專案是一間圖書館，它同時存在兩套服務系統：

- 「檔案模式」(File-Based) — 像「個人借閱書架」。你上傳一份 PDF，系統把它切成書籤(chunks)，要找東西就在這一本書裡翻。每本書是獨立索引。
- 「技能模式」(Skill-Based) — 像「主題館藏」。你建一個叫『刑法』的 Skill，把多本相關 PDF 都掛在它底下，查詢時會跨整個主題館藏一起找。Skill 是後來加上去的，且預期會成為長期主力。

CLAUDE.md 中特別強調這兩套系統「物理隔離」——不同資料夾、不同 SQLite DB、不同 FAISS 索引。換句話說，兩個圖書館員在同一棟樓裡上班，但用各自的書架和帳本，互不打擾。這是好設計(風險隔離)，也是負擔(很多東西被複製了兩份)。

1.3 一張圖看懂高階架構

由上而下、由前而後的五層：



1.4 兩個世界並存:你必須記住的差別

維度	差別
入口頁面	File: /SinglePDFQuery (index.html) Skill: /skill (skill_main.html)
主要 API 前綴	File: /api/v1/chat, /api/v1/upload Skill: /api/v1/skills/*
核心資料表	File: docai.db (documents, chunks) Skill: skill_metadata.db (skill_heads, skill_metadata, skill_chunk_metadata)
向量索引位置	File: /data/faiss_indices/files/ Skill: /data/faiss_indices/skills/
chunk 大小慣例	File: 500 字元 Skill: 1000 字元(或 page-based, 法律文件按頁切)
RAG 流水線	File: 完整 5-階段 OPMP (chat.py 內 chat_stream) Skill: 進化中, SkillServices/progressive_skill_streaming/ 已有 5-phase 雛形
主要使用情境	File: 單檔問答、demo、實驗 Skill: 主題知識庫、跨檔查詢、正式產品

1.5 後端啟動流程

從 main.py(476 行)看入口生命週期:

```
# main.py (節錄)
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 1. 建必要目錄 (uploads, data, logs, static)
    for d in [UPLOAD_DIR, PDF_UPLOAD_DIR, "data", "logs", "static"]:
        Path(d).mkdir(parents=True, exist_ok=True)

    # 2. 初始化 SQLite (檔案版 metadata + WAL 模式)
    file_metadata_provider = await get_file_metadata_provider()

    # 3. 檢查 Milvus(目前 demo 模式禁用,改用 FAISS)
    # 4. 啟動 background_integrity_checker
    #    -> 週期性檢查 FAISS index 是否與 metadata DB 對得起來
    start_background_checker()

    yield # ===== 應用運行中 =====

    # Shutdown
    await file_metadata_provider.close()
```

```
app = FastAPI(lifespan=lifespan)
app.include_router(api_v1_router, prefix="/api/v1")
app.mount("/static", StaticFiles(directory="static"))
```

關鍵字看點:lifespan 是 FastAPI 0.93+ 推薦的取代 on_event("startup/shutdown") 的寫法; WAL(Write-Ahead Logging)是 SQLite 的並發友善模式,系統在多執行緒讀寫 metadata 時靠它存活。

Part 2 · 五大階層細部解剖 (The Five Layers)

2.1 Entry Point — main.py (476 行)

功能單一:組裝 FastAPI app、註冊路由、註冊 lifespan、提供 / 和 /skill 兩個首頁。本檔尺寸合理,不需要拆。

2.2 API Layer — app/api/v1/endpoints/

這是「對外的窗口」。每一個 endpoint 檔案應該只負責:(a) 校驗請求、(b) 呼叫 service、(c) 包裝回應。理想上很薄,但這個專案的 endpoint 並不薄,反而塞了大量商業邏輯(這就是後面要拆檔的主因)。

檔案	行數	主要職責
skills.py	5,581	Skill CRUD、PDF 上傳、串流問答、匯出/匯入、Session 管理。內含 ~30 個 endpoint。
chat.py	894	File-based 5 階段 OPMP 串流問答、單檔摘要。
upload.py	829	File-based PDF 上傳:同步與 SSE 串流兩種。
router.py	779	把所有 endpoint 串起來掛到 /api/v1。
settings.py	264	系統設定讀寫。
llm.py	263	LLM provider 切換、key 管理。
files.py	252	檔案 metadata 管理。

2.3 Service Layer — app/Services/ (File-Based 核心邏輯)

負責「怎麼處理問題」。注意這裡只服務 File-Based 系統,Skill-Based 邏輯放在 SkillServices/。

檔案	行數	主要職責
iterative_query_expansion_service.py	1,010	把使用者問題擴展成多個子問題以提高 recall。
input_data_handle_service.py	677	PDF/文字 前處理、清洗、語言偵測。
chunking_strategies.py	579	切塊策略:recursive、semantic、token、page-based。
prompt_service.py	466	提示詞模板管理 (今晚改過的就是這檔)。
retrieval_service.py	314	檢索與重排序。
query_enhancement_service.py	312	正規化查詢。
query_intent_detection/*	~1,100 合計	查詢意圖偵測(strategy 模式,有 regex/nlp/string)。

2.4 SkillServices Layer — app/SkillServices/ (Skill-Based 核心邏輯)

Skill 系統獨立的服務層。注意目錄名是大寫 S 開頭 — 命名上其實有點奇怪 (Python 慣例是 snake_case), 這是技術債候選。

檔案	行數	主要職責
pdf_skill_ingestion_service.py	1,366	按「頁」切塊的 PDF 攝取流水線(專為法律文件設計)。
progressive_skill_streaming/phase1~5*.py	~2,500 合計	Skill 系統的 5-phase OPMP 雛形。
skill_ingestion_service.py	414	通用 Skill 攝取(用 chunking_strategies)。
base_retrieval.py	240	Retrieval 抽象基類。
index_integrity.py	164	FAISS / metadata 一致性檢查。
skill_retrieval_service.py	160	Skill 檢索實作 (FAISS 後端)。
background_integrity_checker.py	105	啟動時的背景檢查任務。

2.5 Provider Layer — app/Providers/

這是與「真實世界」對話的薄抽象層。Provider 模式的核心:每個外部依賴都有 client.py(實作)+ __init__.py(導出 + factory)。

Provider	client.py 行數	包了什麼
skill_metadata_provider	2,030	SQLite (skill_heads/skill_metadata/document_mapping/chunk_metadata/processing_jobs)。
vector_store_provider	733 (+ 536 milvus)	FAISS / Milvus 雙後端。
file_metadata_provider	643	SQLite (documents, chunks, chunk_metadata)。
chat_history_provider	512	MongoDB 對話紀錄。
cache_provider	492	Redis, 沒 Redis 就退回 in-memory。
llm_provider	298 + 293 (manager)	Claude / GPT / Gemini 切換。
embedding_provider	219 + 208 (bge)	OpenAI Embedding 或本地 BGE-M3。

Singleton + lazy 是這層的標準寫法 — 每個 provider 都暴露一個 get_xxx_provider() 工廠函式, FastAPI 透過 Depends(get_xxx_provider) 注入。

Part 3 · 兩條主要資料流 (Data Flow Walkthroughs)

3.1 攝取流程 (Ingestion): 一份 PDF 是怎麼變成可查的知識?

以 Skill 系統為例(File 系統流程大同小異,只是落點不同):

```
# 偽碼:用戶上傳 PDF 到 Skill 的旅程
async def upload_source_to_skill(skill_id, pdf_file):
    # —— Step 1. API 層收到 multipart upload ——
    # endpoints/skills.py :: upload_source_to_skill()
    pdf_path = save_pdf_to_disk(pdf_file, dir=settings.PDF_UPLOAD_DIR)

    # —— Step 2. 啟動處理任務 (有同步與 SSE 串流兩條路) ——
    # 流水線進入 PDFSkillIngestionService
    result = await pdf_ingestion_service.process_pdf_streaming(
        pdf_path=pdf_path, head_id=skill_id,
    )
    # 在 SkillServices/pdf_skill_ingestion_service.py 內:
    #
    # 2a. _extract_pdf_pages → PyPDF2 逐頁讀文字
    # 2b. _clean_page_text → 去頁碼/頁眉/亂碼
    # 2c. _page_generator → 變成 async generator
    # 2d. (對每 batch)
    #         embedding_provider.embed_batch(texts) # BGE-M3, 1024 維
    #         vector_store.add_vectors(vectors, store_type="skill")
    #         skill_metadata_provider.create_skill( # 子文件 metadata
    #             skill_id=child_doc_id,
    #             head_id=skill_id,
    #             source_name=pdf_path.stem,
    #             ...)
    #         skill_metadata_provider.upsert_chunk_metadata(...)
    # 2e. processing_job 表追蹤進度,可以中斷後重啟 (startup_recovery)

    # —— Step 3. 結果 ——
    return {
        "pages_extracted": result.pages_extracted,
        "chunks_created": result.chunks_created,
        "status": "success",
    }
```

攝取結束後,FAISS 上多了 N 條向量,SQLite 上多了 N 條 chunk metadata + 1 條 document 紀錄; Skill 的總 chunk 數也會跟著更新。

3.2 查詢流程 (Query): 一個問題是怎麼變成答案?

以 File-Based chat 的 OPMP 5 階段流水線為例 — 這套已經實作完整 SSE 串流;Skill-Based 的對應版本還在 progressive_skill_streaming/ 累積中。

```
# 偽碼:一個問題的旅程 (File-Based OPMP)
async def chat_stream(request: ChatRequest):
    # endpoints/chat.py :: chat_stream() (~453 行)

    # —— Phase 1: Query Understanding ——
    # 意圖偵測 + 查詢擴展 (iterative_query_expansion_service)
    expanded = await expansion_service.expand_iteratively(request.query)
    yield sse_event("phase", {"name": "understanding", "progress": 20})

    # —— Phase 2: Parallel Retrieval ——
```

```

# 對每個子問題, 並行打 FAISS, 取 top-k
all_hits = await asyncio.gather(*[
    retrieval_service.search(q, top_k=5) for q in expanded.queries
])
chunks = deduplicate_and_rank(flatten(all_hits))
yield sse_event("phase", {"name": "retrieval", "progress": 50})

# — Phase 3: Context Assembly —
# 拼成給 LLM 的 prompt, 加入引用標記
prompt = prompt_service.build_rag_prompt(
    question=request.query, chunks=chunks, lang="zh-TW",
)
yield sse_event("phase", {"name": "assembly", "progress": 70})

# — Phase 4: Response Generation —
# 串流呼叫 LLM, 逐 token 吐出
async for token in llm_provider.stream_chat(prompt):
    yield sse_event("token", token)

# — Phase 5: Post-Processing —
# 寫對話歷史到 MongoDB, 留統計
await chat_history.save(session_id, request.query, full_answer, chunks)
yield sse_event("done", {"progress": 100})

```

「OPMP」= Optimistic Progressive Markdown Parsing: 前端在 phase 4 拿到 token 時就一邊渲染 Markdown, 等到完成時通常已經漂亮地排好版了。實作見 `static/js/progressive_markdown_renderer.js` (513 行)。

Part 4 · 速查術語 (Glossary)

術語	意思
RAG	Retrieval-Augmented Generation。先檢索再讓 LLM 用檢索結果答題。
Chunk	把長文切成的小段。本專案 File 系統 ~500 字、Skill 系統 ~1000 字或按頁。
Embedding	把文字轉成向量(這裡是 1024 維)。用 BGE-M3 或 OpenAI text-embedding-3。
BGE-M3	智源研究院的多語 embedding 模型,中文表現好,本地跑省 API 費用。
FAISS	Facebook 出的向量索引,單檔、快、無需服務。Demo 用這個。
Milvus	向量資料庫服務,適合大規模。目前 demo 禁用。
SSE	Server-Sent Events:HTTP 的單向串流,適合進度回報、串流答案。
OPMP	Optimistic Progressive Markdown Parsing,邊串流邊渲染 Markdown。
Provider	本專案的抽象層命名:Llm/Embedding/VectorStore 等的 wrapper。
Skill Head / Skill / Document	Skill 系統的三層階層: Head 是定義 (= 一個主題)、Document 是其下的 PDF、chunks 屬於 Document。
WAL	SQLite 的並發友善寫入模式,本專案用它讓 metadata 在多執行緒下穩定。

Part 5 · 動手:具體 Python 範例

以下範例都對齊現有 API 介面,適合你打開 Python REPL / 寫個 ad-hoc script 跑跑看,把概念落地。

5.1 開啟一個 Provider:取得 Skill Metadata Provider

```
import asyncio
from app.Providers.skill_metadata_provider import get_skill_metadata_provider

async def main():
    provider = await get_skill_metadata_provider()
    heads = await provider.list_skill_heads()
    for h in heads:
        print(h["head_id"], h["skill_name"])

asyncio.run(main())
```

Pattern 重點: 永遠用 `await get_xxx_provider()` 取 — 它是 singleton + lazy,第一次會做 DB 初始化 (建表、開 WAL、寫 schema_version)。

5.2 走完一個小型 RAG 迴圈

```
import asyncio
from app.Providers.embedding_provider import get_embedding_provider
from app.Providers.vector_store_provider import get_vector_store_provider
from app.Providers.llm_provider import get_llm_provider

async def quick_rag(question: str, head_id: str):
    embed = await get_embedding_provider()
    vstore = await get_vector_store_provider()
    llm = await get_llm_provider()

    # 1. embed the question
    q_vec = (await embed.embed_batch([question]))[0]

    # 2. search the skill index
    hits = await vstore.search(
        store_type="skill", store_id=head_id,
        query_vector=q_vec, top_k=5,
    )

    # 3. compose prompt
    context = "\n\n".join(h["text"] for h in hits)
    prompt = f"以下是參考資料:\n{context}\n\n問題: {question}\n請用中文回答。"

    # 4. ask LLM
    answer = await llm.complete(prompt)
    return answer, hits

# asyncio.run(quick_rag("什麼是 LLM 預訓練?", "head_xxxx"))
```

5.3 觀察 endpoint 是怎麼被掛起來的

```
# app/api/v1/router.py (簡化版)
from fastapi import APIRouter
from app.api.v1.endpoints import skills, chat, upload, files, llm, settings
```

```

router = APIRouter()
router.include_router(skills.router, prefix="/skills", tags=["skills"])
router.include_router(chat.router, prefix="", tags=["chat"])
router.include_router(upload.router, prefix="", tags=["upload"])
router.include_router(files.router, prefix="/files", tags=["files"])
router.include_router(llm.router, prefix="/llm", tags=["llm"])
router.include_router(settings.router, prefix="/settings", tags=["settings"])

```

注意: Skill 的所有 URL 都長 `/api/v1/skills/...`; File 的 `/chat` 和 `/upload-pdf` 為了向後相容沒有 prefix。

5.4 一個典型 endpoint 的形狀(讀懂 `skills.py` 的單元)

```

# 從 endpoints/skills.py 的 list_skills() 一窺:
@router.get("/", response_model=List[SkillResponse])
async def list_skills(
    metadata_provider: SkillMetadataProvider = Depends(get_skill_metadata_provider),
):
    """List all skills with their basic info."""
    try:
        skills = await metadata_provider.list_skills()
        return [SkillResponse(**s) for s in skills]
    except Exception as e:
        logger.error(f"list_skills failed: {e}")
        raise HTTPException(500, detail=str(e))

```

形狀很乾淨;但問題是這份檔案裡有 ~30 個這樣的 endpoint, 加上幾個極度肥大的 (`stream_skill_chat` 一個就佔 ~380 行)。下一部分會逐一拆解。

Part 6 · 這個專案的優點 (Strengths)

6.1 Provider 抽象做得不錯

LLM / Embedding / VectorStore 都有清楚的抽象介面,要換 Claude 換 GPT、換 FAISS 換 Milvus、換 OpenAI Embedding 換 BGE 都只動 1~2 個檔。這是長期維護最值錢的設計決策。

6.2 兩個系統「物理隔離」

File-Based 與 Skill-Based 用不同資料夾、不同 DB、不同 FAISS 索引。Skill 系統大幅重構時不會壞掉 File 系統。對 demo 與漸進演進是好設計。

6.3 Single Source of Truth(已修正)

2025-12-04 那次重構把 skill_config.json 廢掉,改用 skill_heads 表當唯一定義來源。這是健康的反模式清理。

6.4 完整的 OPMP 串流體驗 (File-Based)

5 階段 SSE + 漸進式 Markdown 渲染,使用者體感優於多數開源 RAG demo。Skill 端也已有 5-phase 雛形可以複用。

6.5 處理長 PDF 的「Stream-Batch-Persist」架構

pdf_skill_ingestion_service.py 用「邊讀邊批次 embed + atomic batch persist + processing_job 表」,即使 1000+ 頁的 PDF 中途死掉也能 startup_recovery 接著做。這是企業級設計。

6.6 CLAUDE.md 有完整的「修改日記」

從 11/26 到 12/16 的每一次重要修改都有記錄、原因、回滾方案。這是極稀有的紀律。

6.7 索引完整性的 background checker

background_integrity_checker.py 在啟動時跑,定期驗證 FAISS 與 metadata DB 對得起來,出問題能自動回報。這是生產級品質。

Part 7 · 缺點與技術債 (Weaknesses & Tech Debt)

以下按嚴重程度排序。第一名遠遠超過其他項。

7.1 ★★★★★ 巨型檔案 (File-Size Monstrosities)

用「金字塔」描述比較貼切：

檔案	行數	症狀
template/skill_main.html	6,600	HTML + 內嵌 CSS + 內嵌 JS。改 UI 像拆炸彈。
app/api/v1/endpoints/skills.py	5,581	30 個 endpoint + 多個 ~200 行的 helper 全擠一檔。
template/skill_config.html	5,204	同上,UI 邏輯與樣式糾纏。
app/Providers/skill_metadata_provider/client.py	2,030	一個類別承擔 5 個資料表的所有 CRUD + migration + job mgmt。
app/SkillServices/pdf_skill_ingestion_service.py	1,366	_process_large_pdf 一個 method 就 317 行。
template/index.html	1,164	另一個 UI 大檔。
scripts/init_docker_windows.py	1,060	Windows 初始化 + GPU 偵測 + 全部混在一起。
app/Services/iterative_query_expansion_service.py	1,010	一個 class 內塞了 strategy + 評分 + 意圖偵測。

這是當前最大的技術債。Part 8 給每一份檔案具體的拆分方案。

7.2 ★★★★★ 命名與目錄風格不一致

- 目錄: Providers/, Services/, SkillServices/ 用大寫開頭 — Python 慣例是小寫(providers/、services/)。
- 檔案: skill_metadata_provider/client.py、vector_store_provider/client.py — 多個 provider 的「主檔」都叫 client.py,在編輯器 tab 列上全長一樣,認知負擔高。
- 備份: template/bak/ 內有 skill_main.html (3,147) 與 skill_config.html (2,400) 的舊版佔據版本控制 — 應該交給 git 管,不是 bak/ 資料夾。

7.3 ★★★★★ 內嵌 Pydantic Models

光 skills.py 一份檔案內就定義了 18 個 Pydantic models(SkillCreateRequest、SkillResponse、AddSourceRequest...),每一個都散落在不同行(91、102、115、136、144、154、161、169、2974、3029、3036、3450、3459、3579、3904、3913、4163)。這讓 import、跨檔複用、測試模型本身都麻煩。chat.py 也有同樣問題。

7.4 ★★★★★ Endpoint 內塞商業邏輯

stream_skill_chat()(~380 行)、_process_large_pdf()(~317 行)、_producer_loop()(~200 行)都把流水線編排寫死在 endpoint/service 內,呼叫 LLM、寫 DB、發 SSE 一條龍。測試難寫(必須 mock 半個世界)、重用差(File 與 Skill 各自又寫一遍類似邏輯)。

7.5 ★★★★★ Service 層職責邊界模糊

- File 與 Skill 兩條 RAG 流水線各自實作 5 階段,有大量「形狀類似但實作不同」的程式碼。
- `iterative_query_expansion_service` 裡同時放了 `strategy enum`、評分函式、意圖偵測 helper,在概念上應該各自獨立。

7.6 ★★ SQL 寫死、無 migration 框架

`skill_metadata_provider/client.py` 的 `initialize_database` 是 ~195 行硬寫的 `CREATE TABLE / CREATE INDEX`。後續加欄位靠 `scripts/migrate_*.py` 一支一支寫。如果改用 Alembic 或 yoyo-migrations 會省很多力氣。

7.7 ★★ Prompt 與設定硬編在 class 內

`iterative_query_expansion_service.py` 內有大段 `ROUND1_PROMPT`、`ROUND2_PROMPT`、`SUMMARY_KEYWORDS` 都是 class-level 常數。提示詞調整必須改 `.py` 檔重啟服務,而非熱載入 YAML/JSON。

7.8 ★ Scripts 與正式碼界線不清

`scripts/` 內有 22 個 `.py`,有些是一次性 migration、有些是常駐工具、有些是測試。最大的 `init_docai_windows.py` (1,060) 已經是「半套配置管理」,應該抽到 `app/bootstrap/` 或拆成多支。

7.9 ★ 缺乏自動化測試

`test/` 目錄只有 351 行;`scripts/test_*.py` 是 ad-hoc 而非 pytest 套件。缺 CI、缺 coverage 報告,所有「Demo 已就緒」的紀錄都是人工驗證 — 拆檔重構時這是最大風險。

Part 8 · 巨型檔案拆分計劃

拆分四原則(整本指南都以這四點為判準):

- 高內聚 (High Cohesion): 同檔內的函式應該為「同一件事」而存在(同一個資源、同一個生命週期、同一個變更原因)。
- 低耦合 (Low Coupling): 跨檔的依賴越少越好, 理想是「單向 import, 不互相 import」。
- 保留語意完整性: 不為了行數而拆 — 一切到一半的 transaction、共享的 closure state、緊密的 async generator 要整體搬。
- 可漸進、可回退: 每次拆 1 個檔, 確認 server 起得來、demo case 過得了, 才動下一個。

拆分順序按「風險由低到高」, 讓你越拆越有信心。

8.A 最容易拆: `iterative_query_expansion_service.py` (1,010 行)

這個檔案結構好, 內聚度高, 只是「住在同一個 class 裡的東西其實是三個獨立概念」。拆它幾乎沒有風險。

為什麼可以拆

ExpansionStrategy enum、ExpandedQuery dataclass、ExpansionResult dataclass 是純資料容器; intent 偵測(detect_summary_intent 等)只用 regex/keywords, 完全獨立; ROUND1/ROUND2 提示詞是常數。

建議拆分後目錄

```
app/Services/query_expansion/
├── __init__.py          # 對外只 re-export IterativeQueryExpansionService 與
factory
├── models.py            # ExpansionStrategy, ExpandedQuery, ExpansionResult (~40
行)
├── prompts.py           # ROUND1_PROMPT, ROUND2_PROMPT, SUMMARY/MULTI_FILE
keywords (~80 行)
├── intent.py            # detect_summary_intent / detect_multi_file_intent /
*_legacy (~150 行)
├── scoring.py           # _score_queries / _select_best_query /
_assess_query_complexity (~100 行)
├── service.py           # IterativeQueryExpansionService - 純 orchestrator (~400
行)
└── factory.py           # get_iterative_query_expansion_service() (~30 行)
```

內聚/耦合判斷

- **models / prompts / intent / scoring** 彼此互不 import, 只有 service.py 同時 import 它們 — 完美的單向依賴。
- **風險: 極低。** 所有 importer 只需改 from app.Services.iterative_query_expansion_service import ... → from app.Services.query_expansion import ..., 可以在 __init__.py 同時 re-export 維持向後相容。

執行步驟

- step 1: 在 query_expansion/__init__.py 加 from .service import IterativeQueryExpansionService, get_iterative_query_expansion_service。

- step 2: 在舊檔加 `from app.Services.query_expansion import *` 做臨時別名,觀察是否所有 `import` 還能用。
- step 3: 一支一支搬 (先搬 `models`,再搬 `prompts`,再搬 `intent`,再搬 `scoring`,最後 `service`)。
- step 4: 刪除舊 `iterative_query_expansion_service.py`,搜尋專案內所有 `import` 並更新路徑。

8.B 第二容易: upload.py (829 行)

這個檔案有兩個 endpoint(同步上傳、SSE 串流上傳),它們共享一個 processing pipeline 但各自把流水線寫了一次。

建議拆分後目錄

```
app/api/v1/endpoints/upload.py      # 只保留 upload_pdf, upload_file_stream 兩個 route handler (~200 行)
app/Services/file_ingestion/
├── __init__.py
├── file_io.py                      # save_uploaded_file (~40 行)
├── processor.py                   # process_and_embed_file (~180 行)
├── streaming_processor.py          # process_file_streaming + SSE generator (~220 行)
└── sse_helpers.py                 # create_upload_sse_event (與 chat/skills 共用)
```

內聚/耦合判斷

- `processor.py` 與 `streaming_processor.py` 內聚高:都是「處理一份檔案」的兩種變體。
- `sse_helpers.py` 是橫切關注點,目前重複出現在 `chat.py`、`upload.py`、`skills.py`。抽到 `app/utils/sse.py` 三邊共用。
- 風險:中。BackgroundTasks 注入 + Path 物件需要小心搬;有些 closure 與 settings 綁定。

8.C 第三優先: chat.py (894 行)

這個檔案的問題是 `chat_stream()` 一支 ~453 行的 `async generator`,5 個 phase 全壓在同一個 `try/finally` 內。`chat_non_streaming` 又把幾乎相同的邏輯寫了一次(~205 行)。

建議拆分後目錄

```
app/api/v1/endpoints/chat.py        # 兩個 route handler + sequential_document_summary (~200 行)
app/Services/opmp_pipeline/
├── __init__.py                     # 對外: run_opmp_stream(req) -> async gen,
├── run_opmp_sync(req) -> dict
├── models.py                       # ChatRequest, ChatResponse,
SequentialSummaryRequest/Response
├── phase1_understanding.py         # 呼叫 query_expansion + intent detection
├── phase2_retrieval.py             # 並行向量檢索 + 去重排序
├── phase3_assembly.py              # 組 prompt + 引用標記
├── phase4_generation.py            # LLM 串流 token (yield)
├── phase5_postprocess.py           # MongoDB 寫對話歷史 + 統計
└── orchestrator.py                # 串起 phase1~5,提供 stream / sync 兩種視角
```

內聚/耦合的細節

- Phase 1~5 模組每個只認自己的 input/output 型別(在 models.py 內定義),用 yield 傳遞中間結果。
- orchestrator 是唯一同時 import 5 個 phase 的地方 — 維持單向、線性的相依鏈。
- chat_stream() 與 chat_non_streaming() 的 95% 重複邏輯就此消除 — sync 版只是 stream 版的 await 全收。
- 風險:中-高。OPMP 是這個專案最有價值的特性,測試覆蓋率又低,拆它必須先寫對應的 phase-level test。

與 Skill 系統的綜效

一旦 OPMP pipeline 被抽出,SkillServices/progressive_skill_streaming/ 那 5 支 phase 檔可以共用 phase3/phase4/phase5,只在 phase1/phase2 內注入 Skill 特定的檢索 strategy。長期消除「File / Skill 兩條流水線」的重複。

8.D 第四優先: pdf_skill_ingestion_service.py (1,366 行)

一個 class 包了:PDF 文字抽取、文字清洗、批次 embedding、原子寫入(FAISS + SQLite)、processing_job 表管理、startup recovery。最危險的是 _process_large_pdf 一支 317 行。

建議拆分

```
app/SkillServices/pdf_ingestion/
├── __init__.py
├── service.py                # PDFSkillIngestionService - 對外 facade,
                             # 只剩 process_pdf_streaming / process_pdf_directory /
get_job_status (~200 行)
├── extractor.py              # _extract_pdf_pages, _page_generator,
                             # get_pdf_page_count, _clean_page_text (~180 行)
├── batch_persister.py        # _store_embeddings, _store_metadata,
                             # _atomic_batch_persist (~330 行)
├── pipeline.py               # _process_large_pdf - 拆成 4~5 個 step function (~350
行,但拆後沒有 >100 行的單函式)
├── recovery.py               # startup_recovery, cleanup_job (~220 行)
└── checkpoint.py            # processing_job 表 wrapper(可選,也可直接用 provider)
```

拆法重點

- pipeline.py 內把 _process_large_pdf 的 317 行切成: validate_input、init_job、process_batch(loop)、finalize_job、cleanup_on_error 五個 step function。
- batch_persister.py 是真正「原子寫」的地方,它應該擁有 transaction context manager;這樣 pipeline.py 只要 `async with persister.batch(...) as p:` 就能保證原子性。
- recovery.py 與 service.py 都會用 batch_persister.py,但 batch_persister 不該認識它們 — 維持單向依賴。
- 風險:中。async generator 與 checkpoint 的 state 必須整體一起搬,不能切到一半。

8.E 第五優先: skill_metadata_provider/client.py (2,030 行)

經典的「上帝物件」(God Object):一個 SkillMetadataProvider class 同時 CRUD 五張表。但因為它本來就是 provider 抽象的實作,公開介面相對穩定,可以「內部拆,外部不變」。

建議拆分:Mixin 法或組合法二選一

方式一:Mixin(改動最小,但 Python mixin 不見得人人喜歡)

```
app/Providers/skill_metadata_provider/
├── __init__.py          # get_skill_metadata_provider() factory (不變)
├── client.py            # class SkillMetadataProvider(
                        #     _ConnectionMixin, _SkillCrudMixin,
                        #     _HeadCrudMixin, _DocMappingMixin,
                        #     _JobMixin, _MaintenanceMixin):
                        #     pass
                        # 只 30~50 行,純組裝
├── _connection.py       # __init__, _get_connection, _execute_with_cleanup,
close
├── _schema.py           # initialize_database (那 195 行的 CREATE TABLE)
├── _skill_crud.py       # create_skill / get_skill / list_skills / update /
delete
├── _head_crud.py        # skill_heads CRUD + get_skill_tree
├── _doc_mapping.py      # link_skill_to_document + 兩個查詢
├── _overview.py         # store_overview / get_overview /
get_multiple_overviews
├── _processing_status.py # update_processing_status, get_incomplete_skills,
reset_failed_skills
├── _jobs.py             # processing_job 全套 CRUD
├── _chunk_metadata.py   # get_chunk_metadata
├── _migration.py        # migrate_existing_data
└── _maintenance.py      # checkpoint_wal, get_wal_stats
```

方式二:組合(更乾淨,但要動少量 caller — 因為現在 caller 都呼叫 `provider.create_skill()`,改成組合後仍可在 `client.py` 提供同名 method 委派)

```
# client.py
class SkillMetadataProvider:
    def __init__(self, db_path):
        self._conn = ConnectionManager(db_path)
        self.skills = SkillCrud(self._conn)
        self.heads = HeadCrud(self._conn)
        self.docs = DocMapping(self._conn)
        self.overview = OverviewStore(self._conn)
        self.jobs = JobStore(self._conn)
        ...
    # 為了向後相容,維持頂層 method:
    async def create_skill(self, **kw):
        return await self.skills.create(**kw)
    async def list_skill_heads(self, **kw):
        return await self.heads.list(**kw)
    # ... 其他委派
```

內聚/耦合判斷

- 每個 *Crud 類別只用 ConnectionManager,彼此不互 import — 標準單向依賴。
- schema 與 migration 拆分後,改 schema 不用打開 2000 行檔案。
- 風險:中。connection singleton 的 lock 必須整體拿,不能讓兩個 Crud 各自重建。

8.F 最後也是最難: skills.py (5,581 行)

這份檔案是整個專案最大的單一風險。它有 30 個 endpoint, 18 個內嵌 Pydantic models, 1 個 380 行的 `stream_skill_chat` 與 1 個 200 行的 `_producer_loop`。但好消息是它「夠平」——大部分 endpoint 互不相依, 可以按 URL 群組漸進切離。

8.F.1 第一步: 把 models 抽出去(零風險)

```
app/api/v1/endpoints/skills/
├── __init__.py          # router = APIRouter(); 滙入所有 sub-routers
└── models.py           # 全部 18 個 Pydantic models 集中
```

8.F.2 第二步: 按 URL 群組切出 sub-router

```
app/api/v1/endpoints/skills/
├── __init__.py          # router = APIRouter()
                        # router.include_router(crud.router)
                        # router.include_router(config.router)
                        # router.include_router(sources.router) 等
├── models.py
├── _config_io.py        # load_skill_config / save_skill_config 兩個 helper
├── _sse.py              # 共用 SSE helpers
├── crud.py              # GET / POST /, list_skills, create_skill (~120 行)
├── chat.py              # POST /chat, POST /stream/{skill_id} (chat_with_skills
與
                        # stream_skill_chat 的 thin 版, 核心邏輯交給 opmp_pipeline)
├── config.py            # /config/* endpoints (~250 行)
├── sources.py           # /upload-source, /upload-source/stream, /sources/{...}
刪除 (~400 行)
├── pdfs.py              # /pdfs 列舉、/rebuild、/rebuild-threshold、/integrity
(~400 行)
├── instant_attachments.py # /instant-attachments/{...} (~120 行)
├── heads.py             # /heads/* + /tree + /migrate (~270 行)
├── sessions.py          # /sessions/* + acknowledge_phase + delete/clear (~260
行)
├── export_import.py      # /export, /preview-import, /import + sanitize_filename
+ check_zip_safety (~600 行)
├── translate.py         # /translate (~120 行)
└── rename.py            # /rename + /delete (~330 行)
```

8.F.3 第三步: 把流水線抽出去

`stream_skill_chat()` 380 行的內容應該被搬到 `app/Services/opmp_pipeline/`(與 `chat.py` 同一個服務)。Skills 的 stream endpoint 只剩 ~40 行——校驗請求、呼叫 pipeline、把 yield 包成 `EventSourceResponse`。`_producer_loop()` 200 行也應該搬到 `app/SkillServices/pdf_ingestion/pipeline.py` 或新建 `streaming_producer.py`。

8.F.4 為什麼這樣切?

- **URL prefix = 自然邊界:** `/skills/heads/*` 一定是 `heads.py`、`/skills/sessions/*` 一定是 `sessions.py`。沒有解釋成本。
- **內聚由「資源」定義:** 一個檔案內的 endpoint 都操作同一種資源。`export_import.py` 內的 `sanitize_filename/check_zip_safety` 只被該檔用, 留在那裡; `sources.py` 內 `_producer_loop` 太大, 就外移。
- **耦合方向:** 所有 sub-router 都 import `models.py` 與 `_sse.py`; 但 sub-router 之間互不 import。唯一向外的 import 是 `Providers / Services / opmp_pipeline`。

8.F.5 風險與防護

- 風險:高。30 個 endpoint 都被前端 fetch() 寫死 URL,任何 URL 變動會壞 UI。
- 防護 1:保持所有 URL 不變。FastAPI 的 include_router(prefix='') 可以做到「程式碼搬家、URL 不變」。
- 防護 2:第一步只拆 models — 跑一次完整 demo case,確認 OK 再進第二步。
- 防護 3:每次只搬 1 個 sub-router,commit、重啟、跑 demo,再搬下一個。13 個 sub-router = 13 次安全的小 commit。
- 防護 4:為前端打過的最關鍵 endpoint(/skills/, /skills/chat, /skills/stream/{id}, /skills/upload-source/stream)各寫 1 條 integration test,任何 PR 必跑。

8.G HTML 模板的拆分(順帶一提)

skill_main.html (6,600) 與 skill_config.html (5,204) 雖然不是 Python,但問題更嚴重——HTML、內嵌 CSS、內嵌 JS 全在一起。建議:

- 第一刀:把 <style>...</style> 整段挪到 static/css/skill_main.css、static/css/skill_config.css。
- 第二刀:把 <script>...</script> 整段挪到 static/js/skill_main.js、static/js/skill_config.js。
- 第三刀:把 HTML 內可重用片段(sidebar、modal、toast、tree)用 Jinja {% include %} 拆 partial。
- template/bak/ 整個刪掉 — 那是 git 的工作。
- 中期目標:評估上 Vue/React/Alpine 取代手刻 vanilla JS。但這超出本指南範圍。

Part 9 · 拆分執行路線圖 (Refactor Roadmap)

建議分四個 phase,每個 phase 結束時都應該 git tag,且能用 demo case 驗證。

Phase 1 (1-2 天) · Easy Wins

- 拆 `iterative_query_expansion_service.py` → `app/Services/query_expansion/`
- 把 `skills.py` / `chat.py` / `upload.py` 內所有 Pydantic models 抽到各自的 `models.py`(只搬資料,不動邏輯)
- 建 `app/utils/sse.py`,把三邊重複的 `create_*_sse_event` 合一
- 刪除 `template/bak/`(改放 git tag)

Phase 2 (3-5 天) · Provider 內部拆分

- `skill_metadata_provider/client.py` 拆成多 mixin 或組合
- 為 `SkillMetadataProvider` 寫第一批 unit test(用 in-memory sqlite)
- `file_metadata_provider/client.py`(643 行)其實還好,但同款做法可以順手做

Phase 3 (5-7 天) · 攝取流水線拆分

- `upload.py` 內的 processing pipeline 抽出 → `app/Services/file_ingestion/`
- `pdf_skill_ingestion_service.py` 拆出 `extractor` / `batch_persister` / `recovery` / `pipeline`
- 為 `_atomic_batch_persist` 寫專屬 test(它是「掉一片就壞整批」的關鍵點)

Phase 4 (10-14 天) · OPMP + skills.py 大手術

- 把 `chat.py` 的 5 階段抽出 → `app/Services/opmp_pipeline/`
- 讓 `SkillServices/progressive_skill_streaming/` 改用 `opmp_pipeline`,只覆寫 `phase1/phase2`
- `skills.py` 按 URL 群組拆成 sub-routers(13 個 commit,每個都跑 demo)
- 最後:HTML 大檔的 CSS/JS 抽離(但這可以另起 PR)

貫穿整個 Roadmap 的紀律

- 每個 phase 開始前:在 `CLAUDE.md` 加一條「修改日記」(專案已有此慣例,沿用)
- 每個 phase 結束前:確認 demo 案例可跑(用 `CLAUDE.md` 中提到的『刑法』『投資理財』 skill)
- 每動一支 endpoint:用 curl + 前端各驗證一次
- 不確定的拆法,先在 branch 試,不要直接動 `beta_skills_v01`

Part 10 · 附錄 · 建議的學習路徑

如果你只有 3 個半天能用,以下順序最划算:

第 1 個半天

- 讀 `CLAUDE.md`(已通讀)+ 跑一遍 `start_system.sh`,確認本機跑得起來。
- 打開 `main.py` 與 `app/api/v1/router.py` — 看路由全景。

- 在前端 /skill 頁面跑 2~3 個查詢,打開 DevTools Network tab 看 /api/v1/skills/stream/{id} 的 SSE。

第 2 個半天

- 從 endpoints/skills.py 找 list_skills、create_skill、stream_skill_chat 三個 endpoint 順著呼叫鏈讀下去。
- 從 SkillServices/pdf_skill_ingestion_service.py 看 _process_large_pdf 的 stream → batch → persist 結構。
- 跑一次 scripts/test_skill_heads_migration.py 與 scripts/test_direct_faiss.py 觀察底層。

第 3 個半天

- 讀 Part 8 的拆分計劃,選 8.A(query_expansion)動手 — 它最小、最安全。
- 一邊拆一邊跑 demo case,把成功流程加進 test/。
- 結束前到 CLAUDE.md 寫一條修改日記,把這次重構的 why / what / how to rollback 記下來。

祝順利!如果有任何一個區段不清楚,把那段貼回給我,我可以針對該段做更深入的拆解或範例。

